Experiment 9: Multilayer Perceptron Implementation

Question

Problem Statement: A telecom operator wants to predict which customers will cancel their subscription next month. A customer-churn dataset with tenure, monthly charges, contract type, complaints and data usage, with a Churn label. The objective is to build, train and evaluate an MLP classifier in Keras. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Pre-process the data: encode categorical fields and scale numeric fields.
2. Build an MLP with two or three hidden layers using ReLU and a sigmoid output.
3. Train with binary cross-entropy and Adam; plot training and validation curves.
4. Report accuracy, precision, recall, F1 and the confusion matrix.
5. Vary the hidden-layer width and note its effect.

Code

```
"""Experiment 9: Multilayer Perceptron Implementation (Customer Churn)"""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, ConfusionMatrixDisplay)

np.random.seed(42); tf.random.set_seed(42)

# ---------------- Synthetic telecom churn dataset ----------------
n = 3000
df = pd.DataFrame({
    "tenure": np.random.randint(1, 73, n),
    "monthly_charges": np.round(np.random.uniform(20, 120, n), 2),
    "contract": np.random.choice(["Month-to-month", "One year", "Two year"], n, p=[0.5, 0.3, 0.2]),
    "complaints": np.random.poisson(0.8, n),
    "data_usage_gb": np.round(np.random.uniform(1, 50, n), 1),
})
contract_effect = df["contract"].map({"Month-to-month": 1.0, "One year": -0.2, "Two year": -1.2})
logit = -0.3 - 0.05 * df["tenure"] + 0.025 * (df["monthly_charges"] - 65) + 0.5 * df["complaints"] +
contract_effect
df["Churn"] = (np.random.rand(n) < 1 / (1 + np.exp(-logit))).astype(int)
print(df.head()); print("\nChurn rate:", round(df["Churn"].mean(), 3))

# ---------------- Task 1: pre-processing ----------------
X, y = df.drop(columns="Churn"), df["Churn"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
pre = ColumnTransformer([
    ("num", StandardScaler(), ["tenure", "monthly_charges", "complaints", "data_usage_gb"]),
    ("cat", OneHotEncoder(drop="first", sparse_output=False), ["contract"])])
X_tr = pre.fit_transform(X_tr); X_te = pre.transform(X_te)       # fit on train only
print("Processed feature shape:", X_tr.shape)

# ---------------- Task 2: build MLP ----------------
def build_mlp(width=64, n_hidden=3):
    m = keras.Sequential([keras.layers.Input((X_tr.shape[1],))])
    for i in range(n_hidden):
        m.add(keras.layers.Dense(max(width // (2 ** i), 4), activation="relu"))
    m.add(keras.layers.Dense(1, activation="sigmoid"))
    # ---------------- Task 3: BCE + Adam ----------------
    m.compile(optimizer=keras.optimizers.Adam(0.001), loss="binary_crossentropy", metrics=["accuracy"])
    return m

model = build_mlp(64, 3)
model.summary()
```

```
hist = model.fit(X_tr, y_tr, validation_split=0.2, epochs=50, batch_size=32, verbose=0)


fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(hist.history["loss"], label="train"); ax[0].plot(hist.history["val_loss"], label="validation")
ax[0].set_title("Loss"); ax[0].set_xlabel("Epoch"); ax[0].legend()
ax[1].plot(hist.history["accuracy"], label="train"); ax[1].plot(hist.history["val_accuracy"],
label="validation")
ax[1].set_title("Accuracy"); ax[1].set_xlabel("Epoch"); ax[1].legend()
plt.tight_layout(); plt.show()


# ---------------- Task 4: evaluation ----------------
y_pred = (model.predict(X_te, verbose=0).ravel() > 0.5).astype(int)
print(f"Accuracy : {accuracy_score(y_te, y_pred):.4f}")
print(f"Precision: {precision_score(y_te, y_pred):.4f}")
print(f"Recall   : {recall_score(y_te, y_pred):.4f}")
print(f"F1-score : {f1_score(y_te, y_pred):.4f}")
cm = confusion_matrix(y_te, y_pred)
print("Confusion matrix:\n", cm)
ConfusionMatrixDisplay(cm, display_labels=["Stay", "Churn"]).plot(cmap="Blues")
plt.title("Confusion matrix"); plt.show()


# ---------------- Task 5: vary hidden-layer width ----------------
rows = []
for w in [8, 16, 32, 64, 128, 256]:
    m = build_mlp(w, 2)
    m.fit(X_tr, y_tr, validation_split=0.2, epochs=50, batch_size=32, verbose=0)
    pr = (m.predict(X_te, verbose=0).ravel() > 0.5).astype(int)
    rows.append({"Width": w, "Params": m.count_params(), "Accuracy": accuracy_score(y_te, pr),
                 "F1": f1_score(y_te, pr)})
res = pd.DataFrame(rows)
print("\n", res.round(4).to_string(index=False))
plt.figure(figsize=(6, 4))
plt.plot(res["Width"], res["Accuracy"], "o-", label="Accuracy"); plt.plot(res["Width"], res["F1"], "s-",
label="F1")
plt.xscale("log", base=2); plt.xlabel("First hidden-layer width"); plt.title("Effect of width");
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

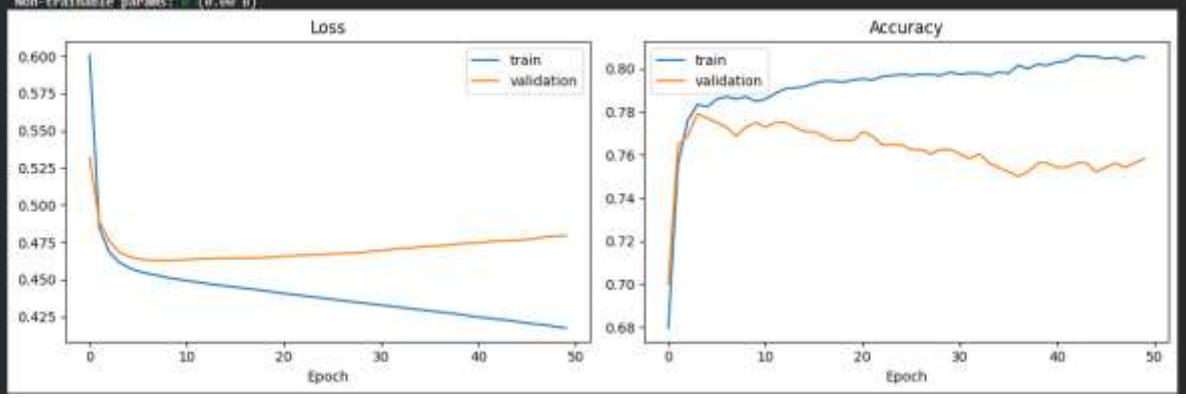
Output

```
   tenure  monthly_charges       contract  complaints  data_usage_gb  churn
0      53           114.30       One year           2            2.3      0
1      15            62.24  Month-to-month          2           28.1      0
2      72            57.73       One year           0           11.4      0
3      61           113.63       One year           0           15.1      0
4      21            87.51       One year           1           45.6      0

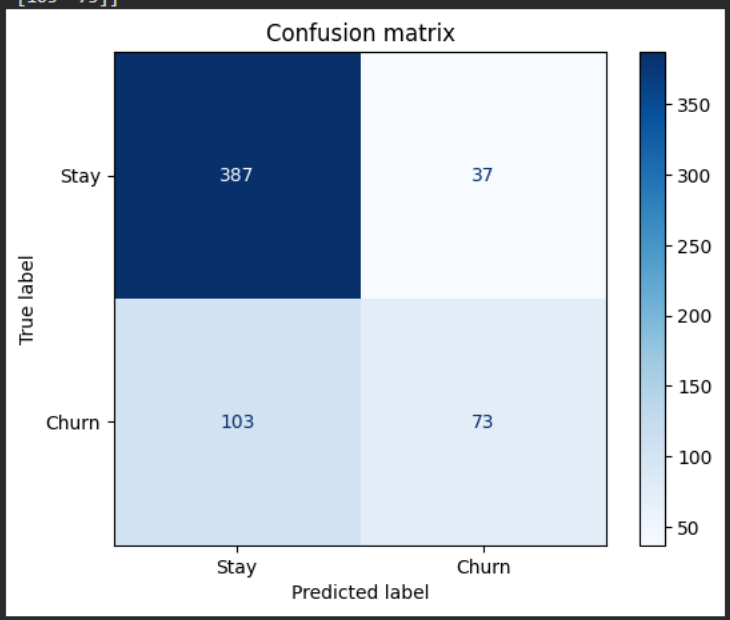
Churn rate: 0.293
Processed feature shape: (2400, 6)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	448
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

```
Total params: 3,073 (12.00 KB)
Trainable params: 3,073 (12.00 KB)
Non-trainable params: 0 (0.00 B)
```




```
Accuracy : 0.7667
Precision: 0.6636
Recall   : 0.4148
F1-score : 0.5105
Confusion matrix:
 [[387  37]
 [103  73]]
```



Width	Params	Accuracy	F1
8	97	0.7617	0.5153
16	257	0.7483	0.4983
32	769	0.7467	0.4899
64	2561	0.7633	0.5035
128	9217	0.7633	0.5069
256	34817	0.7600	0.5068

```
   Width  Params  Accuracy     F1
...    8      97    0.7617 0.5153
      16     257    0.7483 0.4983
      32     769    0.7467 0.4899
      64    2561    0.7633 0.5035
     128    9217    0.7633 0.5069
     256   34817    0.7600 0.5068
```

